КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ТАРАСА ШЕВЧЕНКА
Факультет комп'ютерних наук та кібернетики

Звіт до лабораторної роботи №4

за спеціальністю 122 Комп'ютерні науки

на тему:

ОБРОБКА ВІДЕО НА GPU ІЗ CUDA

Виконав студент

Божевський Іван

КИЇВ 2026

РЕФЕРАТ

У роботі реалізовано конвеєр обробки відео засобами OpenCV і CUDA. Відеофайл декодується послідовно, поточний та попередній кадри передаються до пам'яті GPU, для них обчислюється абсолютна попарна різниця, а результат записується до нового MP4-файлу. Програма не має CPU fallback і потребує OpenCV, скомпільованого з підтримкою CUDA.

Ключові слова: CUDA, GPU, OPENCV, ОБРОБКА ВІДЕО, ПОПАРНА РІЗНИЦЯ КАДРІВ

ЗМІСТ

РЕФЕРАТ 2
ВСТУП 4
1 Теоретичні відомості та алгоритм 5
2 Реалізація програми 6
3 Вхідні дані та середовище 7
4 Методика запуску та результати 8
ВИСНОВКИ 9

ВСТУП

Метою роботи є реалізація обробки відео на графічному процесорі. Завдання включає відкриття відеофайлу за допомогою OpenCV, завантаження кадрів у пам'ять GPU, обчислення попарної різниці кадрів та збереження результуючого відео засобами OpenCV. Така операція є базою для виявлення руху, фоновіднімання та попередньої обробки відеопотоку.

1 ТЕОРЕТИЧНІ ВІДОМОСТІ ТА АЛГОРИТМ

Нехай $I_t(x, y, c)$ --- значення каналу c пікселя з координатами (x, y) у кадрі t. Результат попарної різниці визначається незалежно для кожного пікселя й каналу:

$$D_t(x, y, c) = |I_t(x, y, c) - I_{t-1}(x, y, c)|, \qquad t \geq 1. \tag{1.1}$$

Для першого кадру D_0 записується чорним, оскільки попереднього кадру немає. Операція має часову складність $O(W \cdot H \cdot C)$ на кадр, але її пікселі не залежать один від одного, тому вона природно відображається на GPU.

CUDA --- платформа NVIDIA для виконання паралельних обчислень на GPU. У цій реалізації OpenCV використано лише для `VideoCapture` та `VideoWriter`. Для кожної пари кадрів CUDA Runtime API виділяє буфери пристрою через `cudaMalloc`, копіює дані через `cudaMemcpy`, запускає власний kernel і повертає результат у `cv::Mat` через `cudaMemcpy`. Декодування й кодування лишаються в OpenCV на CPU, тому вимірюваний час повного конвеєра також містить передавання даних CPU--GPU.

Для систем із кількома GPU застосовано data parallelism за парами кадрів. Вхідний потік формує невеликий пакет послідовних пар кадрів. Для кожного CUDA-пристрою створюється C++ потік, який викликає `cudaSetDevice()` та обчислює виділені йому пари. Після завершення потоків кадри записуються у вихідне відео в початковому порядку. Такий поділ не порушує залежність між кадрами I_{t-1} та I_t, бо обидва кадри передаються одному GPU разом.

2 РЕАЛІЗАЦІЯ ПРОГРАМИ

Програму реалізовано мовою C++17 у файлі `src/cuda_video_difference.cpp`; власний CUDA kernel міститься у `src/frame_difference.cu`. CMake компілює цей файл NVCC і підключає `CUDA::cudart`; OpenCV потрібний лише з модулями `core` та `videoio`.

Алгоритм має такі етапи:

1. `cv::VideoCapture` відкриває вхідний MP4 і зчитує його параметри: ширину, висоту та частоту кадрів.
2. До результуючого відео записується чорний перший кадр такого самого розміру.
3. Наступні кадри збираються в пакет, що містить до чотирьох пар на кожен доступний GPU.
4. Кожен GPU у власному потоці виділяє три CUDA-буфери, копіює туди два кадри та запускає kernel, де один CUDA-потік обробляє один байт кадру.
5. Результат копіюється в RAM, CUDA-буфери звільняються, а `cv::VideoWriter` кодує кадри кодеком `mp4v` у правильному порядку.

Перед початком обробки програма перевіряє число пристроїв викликом `cudaGetDeviceCount`. За відсутності CUDA-пристрою вона завершується з пояснювальним повідомленням і не видає CPU-результат за GPU-обробку.

3 ВХІДНІ ДАНІ ТА СЕРЕДОВИЩЕ

Використано файл `16756162_1280_720_25fps.mp4` розміром 7 016 820 байт, що містить 291 кадр роздільністю 1280×720 із частотою 25 кадрів/с. Локальна машина має NVIDIA GeForce RTX 4070 та драйвер версії 621.01. Для запуску встановлено CUDA Toolkit 13.5 і OpenCV 4.14 з FFmpeg. Звичайний пакет `opencv-python` із PyPI не застосовується.

4 МЕТОДИКА ЗАПУСКУ ТА РЕЗУЛЬТАТИ

Програму збирають командами `cmake -S . -B build` і `cmake --build build`, після чого запускають із двома аргументами: шляхом до вхідного та вихідного MP4. Перед відкриттям відео виконується перевірка наявності CUDA-пристрою. Якщо OpenCV не має CUDA-підтримки або GPU недоступний, програма завершується з помилкою; реалізація не виконує обчислення різниці на CPU.

Під час одного локального запуску на RTX 4070 створено результуюче відео розміром 3 409 817 байт. Виміряний час повного конвеєра становить 2.189 с; він включає запуск програми, декодування, ручні копіювання CPU--GPU, CUDA-обчислення, повернення кадру в RAM і кодування MP4. Пропускна здатність становить $291/2.189 = 132.937$ кадрів/с. На цій машині доступна одна відеокарта, тому розподіл виконувався одним GPU-потоком; той самий код автоматично розподілить пакети між усіма GPU на багатопристроєвій системі. Ручне виділення та звільнення пам'яті для кожної пари кадрів додає накладні витрати, але демонструє пряме керування CUDA-пам'яттю.

Таблиця 4.1 --- Результат локального GPU-запуску

Параметр	Значення
GPU	NVIDIA GeForce RTX 4070
CUDA Toolkit	13.5
OpenCV	4.14 з FFmpeg
Кадрів	291
Час повного конвеєра	2.189 с
Пропускна здатність	132.937 FPS
Вихідний файл	`frame_difference.mp4`

Різницевий кадр містить світлі області лише там, де значення пікселів змінилися між двома послідовними кадрами. Статичні ділянки наближаються до чорного, а рухомі об'єкти та зміни освітлення стають помітними.

ВИСНОВКИ

Реалізовано програму CUDA/OpenCV для попарної різниці послідовних кадрів відео. Вона відкриває MP4 через `VideoCapture`, вручну виділяє GPU-пам'ять через `cudaMalloc`, запускає власний CUDA kernel для різниці кадрів, повертає результат через `cudaMemcpy` та записує його через `VideoWriter`. CPU fallback навмисно відсутній: за відсутності CUDA-пристрою програма завершується з помилкою. На RTX 4070 291 кадр оброблено за 2.189 с (132.937 FPS); програма автоматично використовує всі доступні GPU.

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ

[1] OpenCV Documentation. CUDA-accelerated Computer Vision. https://docs.opencv.org/4.x/d2/dbc/cuda_intro.html.

[2] OpenCV Documentation. Video I/O module. https://docs.opencv.org/4.x/dd/de7/group_videoio.html.

[3] NVIDIA. CUDA C++ Programming Guide. https://docs.nvidia.com/cuda/cuda-c-programming-guide/.